

Практическая работа №2. Визуализация данных в Python

1. Цель и задачи работы

Цель работы — освоить статическую и интерактивную визуализацию данных в Python, исследовать влияние параметров t-SNE и UMAP на двумерное представление многомерных объектов и сравнить измеренное время работы алгоритмов.

В работе выполнены следующие задачи:

- загрузка и диагностика многомерного набора Bike Sharing средствами Pandas;
- построение столбчатой и круговой диаграмм с помощью Plotly graph_objects;
- построение линейных графиков с помощью Matplotlib;
- визуализация набора рукописных цифр при разных значениях перплексии t-SNE;
- визуализация тех же объектов UMAP при разных `n_neighbors` и `min_dist`;
- измерение времени и сравнение возможностей библиотек.

2. Исходные данные и первичная диагностика

Основной набор — дневная таблица Capital Bikeshare за 2011–2012 годы. Она содержит 731 наблюдение и 16 исходных столбцов: календарные признаки, сезон, погодные условия, нормированные температуру, влажность и скорость ветра, а также число поездок случайных и зарегистрированных пользователей. Источник: UCI Machine Learning Repository, Bike Sharing Dataset, автор Hadi Fanaee-T.

Таблица 1. Основные группы признаков Bike Sharing

Группа	Поля	Смысл
Идентификатор и дата	instant, dteday	Номер записи и календарная дата
Календарь	season, yr, mnth, holiday, weekday, workingday	Сезон, год, месяц и тип дня
Погода	weathersit, temp, atemp, hum, windspeed	Погодный класс и нормированные измерения
Спрос	casual, registered, cnt	Поездки по типам пользователей и итог

```
df = pd.read_csv("data/day.csv", parse_dates=["dteday"])
df.head()
df.info()
df.isna().sum()
df.duplicated().sum()
```

Таблица 2. Первые пять строк исходного набора

Дата	Сезон	Погода	temp	hum	casual	registered	cnt
2011-01-01	1	2	0.344	0.806	331	654	985
2011-01-02	1	2	0.363	0.696	131	670	801
2011-01-03	1	1	0.196	0.437	120	1229	1349
2011-01-04	1	1	0.200	0.590	108	1454	1562
2011-01-05	1	1	0.227	0.437	82	1518	1600

Таблица 3. Результат первичной диагностики

Показатель	Значение
Размер	731 строк × 16 столбцов
Период	2011-01-01 — 2012-12-31
Пропущенные значения	0
Полные дубликаты	0
Всего поездок	3 292 679
Среднее в день	4504.35
Медиана в день	4548
Максимум в день	8714

Таблица не содержит пропусков и полных дубликатов, поэтому удаление или интерполяция строк не требуются. Проверка `casual + registered = cnt` выполнена для всех 731 наблюдения и не выявила расхождений.

3. Предобработка данных

Дата преобразована в тип `datetime`. Числовые коды сезонов и погоды дополнены читаемыми категориальными названиями. Нормированные погодные показатели переведены в исходные единицы согласно описанию набора.

```

clean["temp_c"] = clean["temp"] * 41
clean["atemp_c"] = clean["atemp"] * 50
clean["humidity_pct"] = clean["hum"] * 100
clean["windspeed_kmh"] = clean["windspeed"] * 67
assert (clean["casual"] + clean["registered"] == clean["cnt"]).all()

```

Очищенная таблица сохранена в CSV и Parquet. Исходные признаки не удаляются: производные поля добавлены отдельно, чтобы сохранить проверяемую связь с источником.

4. Столбчатая диаграмма Plotly

Дневные значения агрегированы по месяцам, чтобы даты оставались читаемыми. Высота и цвет столбца кодируют суммарное количество поездок за месяц. Граница каждого столбца чёрная, толщиной 2; высота интерактивного рисунка равна 700 px.

```

go.Bar(
    x=monthly["month_label"], y=monthly["cnt"],
    marker=dict(color=monthly["cnt"], coloraxis="coloraxis",
                line=dict(color="black", width=2))
)
fig.update_layout(
    title=dict(x=0.5, font=dict(size=20)), height=700,
    margin=dict(l=10, r=10, t=65, b=10)
)
fig.update_xaxes(title_font_size=16, tickfont_size=14,
                 tickangle=315, gridwidth=2, gridcolor="ivory")
fig.update_yaxes(title_font_size=16, tickfont_size=14,
                 gridwidth=2, gridcolor="ivory")

```

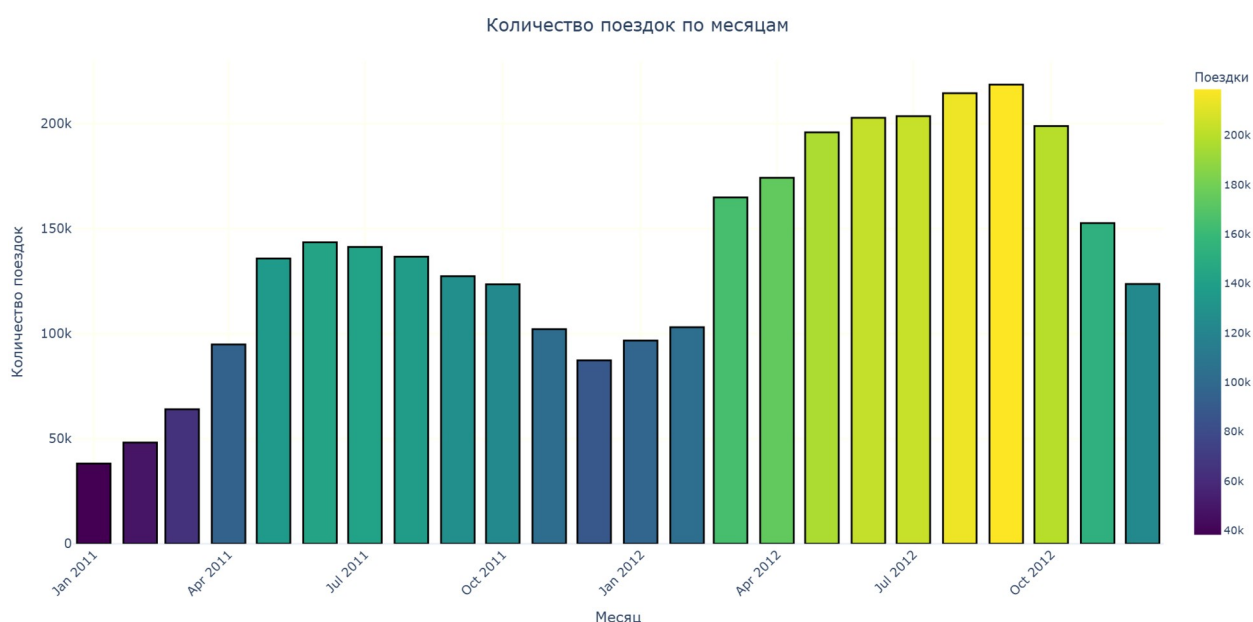


Рис. 1. Количество поездок по месяцам

Пиковый месяц — 2012-09: 218 573 поездок. В 2012 году столбцы в основном выше соответствующих месяцев 2011 года; внутри каждого года виден подъём от зимы к тёплому сезону и спад к концу года.

5. Круговая диаграмма Plotly

Для круговой диаграммы поездки сгруппированы по четырём сезонам. Небольшое число категорий позволяет разместить названия и доли без наложений.

```
go.Pie(  
    labels=season_totals.index,  
    values=season_totals.values,  
    textinfo="label+percent",  
    marker=dict(colors=pie_colors,  
                line=dict(color="black", width=2))  
)
```

Распределение поездок по сезонам

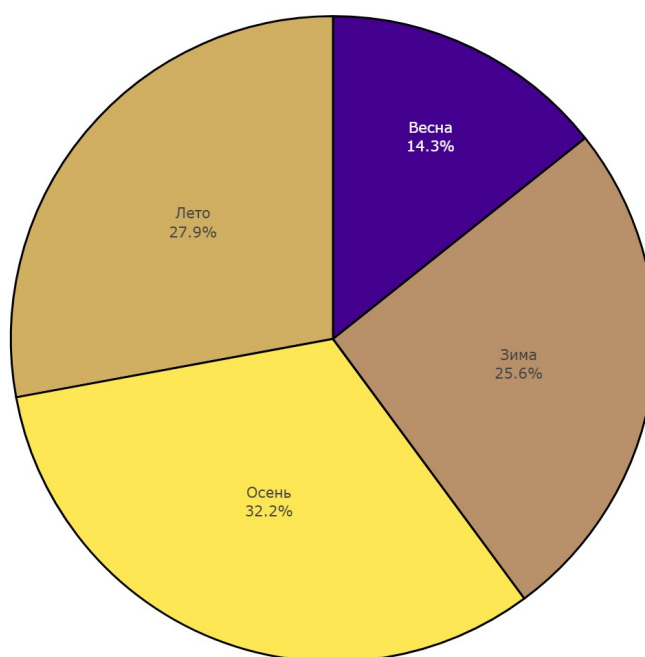


Рис. 2. Распределение поездок по сезонам

Осень формирует наибольший объём — 1 061 129 поездок (32.2%) а весна — наименьший: 471 348 (14.3%).

6. Линейные графики Matplotlib

По общей временной оси сопоставлены три месячных показателя: случайные, зарегистрированные и все поездки. Каждый ряд вынесен в отдельную область

построения и оформлен линией crimson, белыми маркерами с чёрной границей толщиной 2 и сеткой mistyrose толщиной 2.

```
ax.plot(x, y, color="crimson", linewidth=2, marker="o",
        markerfacecolor="white", markeredgewidth=2,
        markeredgewidth=2)
ax.grid(linewidth=2, color="mistyrose")
```



Рис. 3. Динамика спроса по типам пользователей

Все три ряда имеют сходную сезонную форму и заметный рост во втором году. Корреляция месячного общего спроса с зарегистрированными пользователями равна 0.991, со случайными — 0.922. Следовательно, основную динамику общего ряда задаёт зарегистрированная аудитория.

7. Визуализация многомерных данных t-SNE

Для снижения размерности использован встроенный набор `load_digits: 1 797` изображений цифр 0–9, каждое описано 64 интенсивностями пикселей.

Пропуски отсутствуют; признаки стандартизированы. Во всех запусках зафиксирован `random_state=42`, изменялась только перплексия.

```
TSNE(n_components=2, perplexity=perplexity,
      init="pca", learning_rate="auto", max_iter=1000,
      random_state=42).fit_transform(X_scaled)
```

Таблица 4. Результаты t-SNE

Параметры	Время, с	Trustworthiness@10
perplexity=5	1.5213	0.9807
perplexity=30	1.6246	0.9864
perplexity=50	1.9371	0.9857



Рис. 4. t-SNE при разных значениях перплексии

При перплексии 5 локальные группы сильнее дробятся. Значения 30 и 50 дают более цельные кластеры; лучший измеренный показатель сохранения локальных соседств получен при `perplexity=30`. Частичные пересечения объяснимы сходным написанием некоторых цифр. Положение далёких кластеров нельзя трактовать как точное расстояние в исходном 64-мерном пространстве.

8. Визуализация многомерных данных UMAP

UMAP применён к тем же стандартизированным объектам. Уменьшение `n_neighbors` усиливает локальные детали, а уменьшение `min_dist` делает группы компактнее. Чтобы исключить многопоточную невоспроизводимость, использованы `random_state=42` и `n_jobs=1`.

```
umap.UMAP(n_components=2, n_neighbors=n_neighbors,
           min_dist=min_dist, metric="euclidean",
           random_state=42, n_jobs=1).fit_transform(X_scaled)
```

Таблица 5. Результаты UMAP

Параметры	Время, с	Trustworthiness@10
n_neighbors=5, min_dist=0.0	8.4448	0.9780
n_neighbors=15, min_dist=0.1	2.4789	0.9768
n_neighbors=50, min_dist=0.5	3.0352	0.9618

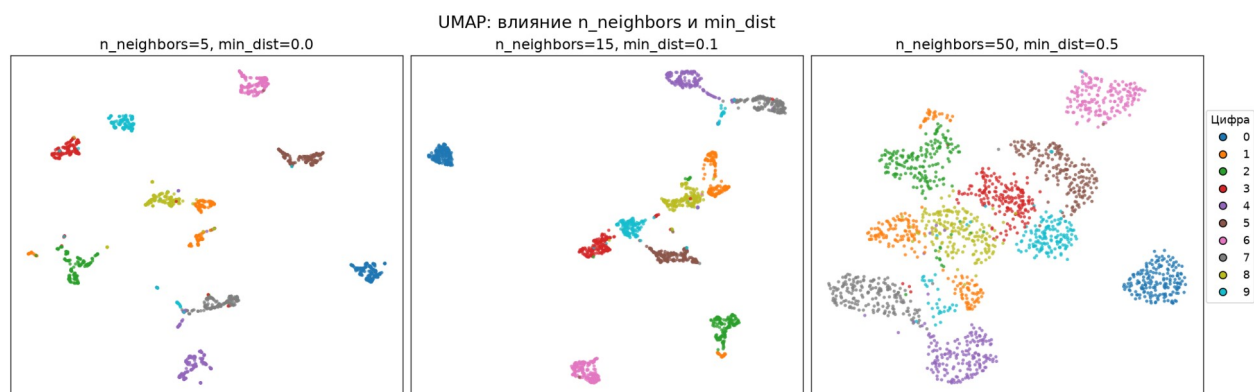


Рис. 5. UMAP при разных $n_neighbors$ и min_dist

Конфигурация $n_neighbors=5$, $min_dist=0$ формирует самые компактные группы. При $n_neighbors=50$ и $min_dist=0.5$ облака становятся шире, а локальная метрика trustworthiness снижается. Поэтому параметры следует выбирать под задачу, а не по визуальной привлекательности одной проекции.

9. Сравнение времени работы

Таблица 6. Измеренное время всех запусков

Алгоритм	Параметры	Время, с	Trustworthiness@10
t-SNE	perplexity=5	1.5213	0.9807
t-SNE	perplexity=30	1.6246	0.9864
t-SNE	perplexity=50	1.9371	0.9857
UMAP	n_neighbors=5, min_dist=0.0	8.4448	0.9780
UMAP	n_neighbors=15, min_dist=0.1	2.4789	0.9768
UMAP	n_neighbors=50, min_dist=0.5	3.0352	0.9618

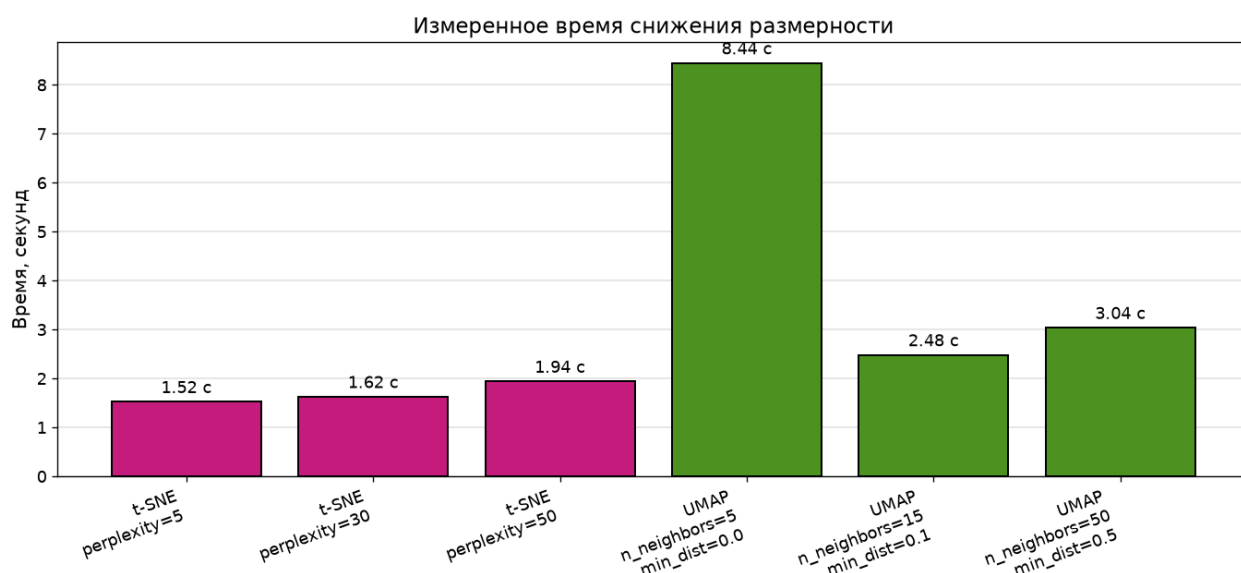


Рис. 6. Сравнение времени снижения размерности

Среднее время t-SNE составило 1.6943 с. Среднее время всех запусков UMAP — 4.6530 с, а двух запусков после первого — 2.7571 с. В данном небольшом наборе t-SNE оказался быстрее. Первый запуск UMAP включает JIT-компиляцию Numba, поэтому результат нельзя обобщать на большие наборы и другие аппаратные среды.

10. Сравнение библиотек

Таблица 7. Сильные стороны и ограничения инструментов

Инструмент	Сильная сторона	Ограничение
Matplotlib	Точный контроль статического рисунка, подграфиков и публикационного экспорта	Интерактивность требует дополнительных средств; много ручных настроек
Plotly	Интерактивные подсказки, масштабирование и автономный HTML	graph_objects многословнее; статический экспорт зависит от браузерного движка
t-SNE	Хорошо подчёркивает локальные соседства и разделение кластеров	Чувствителен к перплексии; глобальные расстояния ненадёжны; нет штатного transform для новых объектов
UMAP	Управляет балансом локальной и глобальной структуры и допускает transform	Результат зависит от параметров; первый запуск несёт расходы JIT-компиляции

11. Итоговые выводы

- Набор Bike Sharing содержит 731 полных дневных наблюдений за два года; пропуски и полные дубликаты отсутствуют.

- За период зарегистрировано 3 292 679 поездок; максимум месячного спроса достигнут в 2012-09.
- Спрос сезонен и во втором году заметно выше; общий месячный ряд теснее связан с зарегистрированной аудиторией.
- Plotly удобен для интерактивного исследования и точных подсказок, Matplotlib — для контролируемой статической композиции.
- t-SNE при perplexity=30 лучше всего сохранил локальные соседства среди проверенных настроек; малое значение сильнее дробит группы.
- UMAP при малых n_neighbors и min_dist создаёт компактные локальные кластеры; рост параметров делает проекцию более разреженной.
- На этом небольшом наборе t-SNE был быстрее UMAP; сравнение времени зависит от JIT-прогрева, объёма данных и окружения.
- Двумерные проекции являются инструментом исследования, а не доказательством истинных кластеров или точных межкластерных расстояний.

12. Ответы на контрольные вопросы

Ниже приведены краткие ответы по объектной модели Matplotlib, возможностям Plotly и интерпретации методов снижения размерности.

1. Из каких основных объектов состоит график Matplotlib?

Простыми словами. График Matplotlib устроен как набор вложенных объектов. Figure — весь лист или холст. Axes — отдельная область, где строятся данные. Axis — конкретная координатная ось X или Y с диапазоном, делениями и подписями. Линии, точки, текст, легенда и другие видимые элементы называются Artist.

Как это работает. Команда `fig, ax = plt.subplots()` создаёт Figure и один Axes. Затем `ax.plot(...)` добавляет линию именно в эту область. У `ax` есть две оси: `ax.xaxis` и `ax.yaxis`. Если написать `plt.subplots(2, 2)`, одна Figure будет содержать четыре Axes, то есть четыре отдельных подграфика.

Пример и вывод. В практической работе рисунок с тремя временными рядами состоит из одной Figure и трёх Axes. Заголовок всего рисунка относится к Figure, а название каждого подграфика — к соответствующему Axes. Такое разделение важно для понимания объектного интерфейса: мы изменяем конкретную часть рисунка, а не «график вообще».

2. Чем интерфейс pyplot отличается от объектного интерфейса Matplotlib?

Простыми словами. pyplot — набор команд, похожий на работу с текущим активным рисунком: `plt.plot()`, `plt.title()`, `plt.xlabel()`. Объектный интерфейс сначала создаёт объекты `fig` и `ax`, а затем явно вызывает их методы: `ax.plot()`, `ax.set_title()`, `ax.set_xlabel()`.

Как это работает. pyplot хранит внутреннее состояние и решает, какой рисунок сейчас считается текущим. Для одного простого графика это удобно и коротко. При нескольких подграфиках легко случайно изменить не ту область. Объектный интерфейс не полагается

на скрытое состояние: переменная `ax` однозначно указывает, куда добавляется линия, сетка или подпись.

Пример и вывод. В цикле можно написать `for ax, column in zip(axes, columns): ax.plot(...)` и настроить каждую панель отдельно. Поэтому для отчётов, сложных композиций и повторно используемых функций предпочтителен объектный интерфейс. `pyplot` всё равно применяется для создания `Figure` и быстрого показа, но основная настройка выполняется через объекты.

3. Зачем явно вызывать `figure()` или `subplots()`?

Простыми словами. Эти функции создают новый рисунок и позволяют заранее определить его размер и устройство. Без явного создания `Matplotlib` может использовать уже активный рисунок, из-за чего новый график случайно наложится на старый или получит неподходящий размер.

Как это работает. `plt.figure(figsize=(8, 5))` создаёт `Figure` размером 8 на 5 дюймов, после чего `pyplot` работает с ней как с текущей. `fig, ax = plt.subplots()` одновременно создаёт `Figure` и `Axes`. Параметры `nrows` и `ncols` задают сетку подграфиков, а `figsize` — физический размер всего рисунка.

Пример и вывод. Для трёх рядов `Bike Sharing` используется `plt.subplots(3, 1, ...)`: три области располагаются вертикально и делят общую временную ось. Явное создание делает код воспроизводимым: размер, число панелей и объект, который изменяется, видны непосредственно в программе.

4. Для чего используется `zorder`?

Простыми словами. `zorder` задаёт, какой объект рисуется поверх другого. Это похоже на слои в графическом редакторе: объект с большим значением находится ближе к зрителю, с меньшим — позади.

Как это работает. Если линия, точки и сетка занимают одно место, порядок их отрисовки влияет на читаемость. Например, сетке можно задать `zorder=0`, столбцам `zorder=2`, а подписям `zorder=3`. Значение не меняет координаты данных, толщину или цвет; оно влияет только на перекрытие элементов.

Пример и вывод. На `scatter plot` крупные точки могут закрывать линию тренда. Если линии задать больший `zorder`, она останется видимой. Параметр особенно полезен для фоновой сетки, выделенных наблюдений, доверительных интервалов и аннотаций. Следует задавать его осмысленно, иначе важные элементы могут оказаться скрытыми.

5. В чём различие `Plotly Express` и `graph_objects`?

Простыми словами. `Plotly Express` — высокоуровневый интерфейс: одна функция быстро строит типовой график из `DataFrame`. `graph_objects` — низкоуровневый интерфейс, где рисунок собирают из отдельных следов и подробно настраивают макет.

Как это работает. В `Express` можно написать `px.bar(df, x="month", y="cnt", color="cnt")`, и библиотека сама создаст `Figure`, следы, легенду и подписи. В `graph_objects` создают `go.Figure()`, добавляют `go.Bar`, `go.Scatter` или другие `trace`-объекты, а затем вызывают `update_layout()` и `update_xaxes()`. Кода больше, зато доступно больше деталей.

Пример и вывод. В этой работе `go.Bar` используется для явной настройки границ столбцов, цветовой шкалы, высоты и полей рисунка. `Express` удобен для разведочного анализа и быстрых графиков. `graph_objects` лучше подходит, когда нужны несколько типов следов, вторичная ось, сложные `hover`-подсказки или точное оформление.

6. Почему Plotly называют интерактивной библиотекой?

Простыми словами. Пользователь может взаимодействовать с готовым рисунком, а не только смотреть на статичную картинку. В браузере доступны наведение мыши, увеличение, перемещение области просмотра, включение и выключение рядов и сохранение изображения.

Как это работает. Plotly преобразует описание Figure в JavaScript-график. Данные и настройки могут быть сохранены в HTML вместе с кодом визуализации. Поэтому для просмотра такого файла не требуется запущенный Python: браузер выполняет интерактивную часть самостоятельно. В notebook график отображается через встроенный HTML/JavaScript-рендерер.

Пример и вывод. На столбчатой диаграмме можно навести курсор на месяц и увидеть точное число поездок, а затем приблизить интересующий период. Интерактивность полезна при исследовании данных. Для печатного отчёта всё равно нужна статическая версия, потому что наведение и масштабирование на бумаге недоступны.

7. Что показывает цветовая шкала в столбчатой диаграмме этой работы?

Простыми словами. Цвет каждого столбца кодирует то же значение, что и его высота: месячное число поездок. Низкие значения показаны одним краем шкалы, высокие — другим. Справа расположена легенда, связывающая цвет с числом.

Как это работает. В `go.Bar` массив `monthly["cnt"]` передан одновременно в `y` и в `marker.color`. Параметр `coloraxis="coloraxis"` подключает общую непрерывную шкалу. Такое кодирование называют избыточным, потому что один показатель выражается двумя визуальными каналами.

Пример и вывод. Высокий жёлтый столбец сентября 2012 года одновременно выделяется высотой и цветом. Это ускоряет поиск максимумов, но цвет не добавляет новый признак. Если цвет должен был показывать год или сезон, в `marker.color` нужно было бы передать соответствующую категорию и использовать дискретную легенду.

8. Почему круговую диаграмму не следует перегружать категориями?

Простыми словами. В круговой диаграмме величины сравниваются по углам и площадям секторов. Человек делает такое сравнение менее точно, чем сравнение длины столбцов. Когда секторов много, они становятся узкими, цвета повторяются, а подписи сталкиваются.

Как это работает. Круг хорошо читается, если категорий мало, их сумма действительно образует целое и различия достаточно заметны. При десяти похожих долях зритель уже не может уверенно определить порядок. Маленькие сектора почти исчезают, а вынесенные подписи занимают больше места, чем сами данные.

Пример и вывод. В практической работе всего четыре сезона, поэтому круговая диаграмма остаётся понятной и показывает долю каждого сезона в общем числе поездок. Если бы категорий было двадцать, лучше использовать отсортированную горизонтальную `bar chart`. Редкие категории иногда объединяют в «Другие», но только если это не скрывает важный смысл.

9. Как выбирать между Matplotlib и Plotly?

Простыми словами. Выбор определяется тем, где и как будет использоваться график. Matplotlib удобен для статических изображений и точного контроля над каждой деталью. Plotly удобен, когда пользователь должен исследовать рисунок в notebook или браузере.

Как это работает. Matplotlib хорошо экспортирует PNG, SVG и PDF, поддерживает сложные научные композиции и обычно предсказуем в печатных документах. Plotly автоматически предоставляет `hover`-подсказки, масштабирование, панорамирование и

управление легендой. За это приходится платить более тяжёлым HTML и зависимостью от браузерного JavaScript.

Пример и вывод. Месячные ряды в отчёте сделаны Matplotlib, потому что важна статическая трёхпанельная композиция. Столбчатая и круговая диаграммы Plotly удобны для просмотра точных значений. Часто используют обе библиотеки: Plotly — на этапе исследования, Matplotlib — для окончательной иллюстрации в отчёте.

10. Какую задачу решает снижение размерности?

Простыми словами. Объект может описываться десятками или тысячами признаков, но человек способен непосредственно увидеть только две или три координаты. Снижение размерности преобразует исходные признаки в меньшее число новых координат, стараясь сохранить важные отношения между объектами.

Как это работает. Алгоритм получает матрицу, где строки — объекты, а столбцы — признаки. На выходе он создаёт, например, две координаты для каждой строки. Разные методы сохраняют разную структуру: PCA ориентируется на общую линейную вариативность, t-SNE — на локальные соседства, UMAP — на граф соседей и сочетание локальной и более общей структуры.

Пример и вывод. Каждая рукописная цифра в `load_digits` описана 64 интенсивностями пикселей. t-SNE и UMAP превращают эти 64 признака в две координаты, чтобы точки можно было показать на плоскости. Проекция помогает увидеть группы и ошибки, но не хранит всю исходную информацию и не заменяет количественную проверку.

11. Что означает перплексия в t-SNE?

Простыми словами. Перплексия приблизительно задаёт, сколько ближайших соседей алгоритм учитывает при описании локального окружения каждой точки. Это не точное число соседей, но полезная интуиция для выбора масштаба анализа.

Как это работает. t-SNE преобразует расстояния между объектами в вероятности соседства. Для каждой точки подбирается ширина локального распределения так, чтобы его эффективный размер соответствовал перплексии. Малое значение сильнее ориентируется на очень близкие точки и может раздробить данные. Большое значение учитывает более широкое окружение и сглаживает мелкие детали.

Пример и вывод. В работе проверены значения 5, 30 и 50. При 5 группы распадаются на более мелкие фрагменты; 30 и 50 дают более цельные классы. Перплексия должна быть заметно меньше числа объектов, а вывод желательно проверять на нескольких значениях, потому что одна картинка может отражать настройку алгоритма, а не устойчивую структуру данных.

12. Почему для t-SNE нужно фиксировать `random_state`?

Простыми словами. t-SNE использует случайность при начальном размещении точек и в процессе оптимизации. Поэтому два запуска с одинаковыми данными и параметрами могут дать повёрнутые, отражённые или иначе расположенные группы.

Как это работает. `random_state=42` задаёт начальное состояние генератора псевдослучайных чисел. При одинаковых версиях библиотек, данных и параметрах запуск становится повторяемым. Число 42 не обладает специальными свойствами; можно выбрать другое фиксированное целое значение.

Пример и вывод. Если сравниваются перплексии 5, 30 и 50, случайное состояние должно быть одинаковым. Иначе различия между рисунками частично объяснялись бы разной случайной инициализацией. Фиксация `random_state` нужна для честного эксперимента и

воспроизводимого отчёта, но не гарантирует совпадение между сильно различающимися версиями библиотек или платформами.

13. Можно ли трактовать расстояния и размеры кластеров t-SNE буквально?

Простыми словами. Нет. t-SNE старается расположить близких соседей рядом, но не обязан сохранять точные расстояния между далёкими группами, их площадь, ориентацию или относительное положение на плоскости.

Как это работает. Для размещения всех точек в двух измерениях алгоритм искажает исходную геометрию. Плотность группы на картинке зависит от параметров и оптимизации. Два кластера могут выглядеть далеко друг от друга, хотя в исходном пространстве это расстояние не имеет такого масштаба. Поворот или зеркальное отражение всей проекции вообще не меняет её смысла.

Пример и вывод. На рисунке цифры 3 и 9 могут находиться рядом из-за похожих пиксельных шаблонов, но расстояние в сантиметрах между их облаками нельзя использовать как числовую меру сходства. Надёжнее анализировать локальных соседей, проверять несколько запусков и использовать метрики в исходном 64-мерном пространстве.

14. За что отвечают `n_neighbors` и `min_dist` в UMAP?

Простыми словами. `n_neighbors` определяет размер окружения, которое UMAP рассматривает вокруг каждой точки. `min_dist` задаёт, насколько близко точки могут располагаться в готовой проекции. Первый параметр управляет масштабом структуры, второй — визуальной плотностью групп.

Как это работает. При малом `n_neighbors` алгоритм сильнее сохраняет очень локальные отношения и может показать много небольших групп. При большом значении учитывается более широкое окружение, поэтому проявляется общая структура. Малый `min_dist` разрешает точкам собираться в плотные кластеры; большой оставляет между соседями больше пространства.

Пример и вывод. В конфигурации `n_neighbors=5`, `min_dist=0.0` цифры образуют компактные островки. При `n_neighbors=50`, `min_dist=0.5` облака становятся шире. Эти параметры не означают «правильное число кластеров» и не выбираются только по красоте: настройки сравнивают по устойчивости, метрике `trustworthiness` и цели анализа.

15. Почему первое измерение времени UMAP может быть завышено?

Простыми словами. Библиотека UMAP использует Numba, которая переводит часть Python-кода в быстрый машинный код во время первого запуска. Компиляция занимает время, поэтому первый вызов включает и подготовку, и само вычисление.

Как это работает. Такой запуск называют «холодным». При следующих вызовах скомпилированные функции уже находятся в памяти или кэше, и измеряется в основном работа алгоритма. На время также влияют загрузка процессора, число потоков, размер данных, параметры и версии библиотек. Поэтому одно число без описания окружения мало что доказывает.

Пример и вывод. В работе первая конфигурация UMAP выполнялась заметно дольше двух последующих. Для честного сравнения полезно отдельно показать холодный запуск, выполнить прогрев, повторить измерение несколько раз и использовать медиану. Нельзя делать общий вывод «UMAP всегда медленнее t-SNE» по одному небольшому набору.

16. Почему двумерная визуализация не доказывает существование истинных кластеров?

Простыми словами. При переходе из 64 измерений в 2 часть информации неизбежно теряется. Алгоритм вынужден раздвигать одни точки и сближать другие. Поэтому на плоскости могут появиться убедительные островки даже тогда, когда в исходных данных нет чётких естественных границ.

Как это работает. t-SNE и UMAP специально оптимизируют вид соседств, а выбранные параметры влияют на плотность и разделение групп. Цветные метки также усиливают впечатление упорядоченности. Настоящий кластер должен подтверждаться не только картинкой, но и устойчивостью к параметрам, метриками качества, результатами других методов и предметным смыслом признаков.

Пример и вывод. Для `load_digits` известны правильные классы цифр, поэтому цвет помогает оценить, смешиваются ли похожие изображения. В реальной задаче меток может не быть. Тогда 2D-проекцию используют как источник гипотез: находят кандидатов на группы, а затем проверяют их в исходном пространстве и на новых данных.